

Syrian Private University

Faculty of Artificial Intelligence Engineering

Department of Systems and Information Security

Project Report

Developing an Intelligent EDR System Based on Natural Language Processing and Deep Learning

Enhancing EDR using NLP and Transformers

Prepared by	Abdulrahman Ahmad Al-Marie Bilal Al-Haffar Omar Kamal
Supervisors	Dr. Wasim Al-Junaidi Dr. Mohammad Al-Shayta
Supervising Engineer	Eng. Abdulrahman Arar

Academic Year 2026/2027

Table of Contents

1. Preface 7
1.1 Importance of the General Specialization: Artificial Intelligence Engineering 7
1.2 Importance of the Specific Specialization: Systems and Information Security 7
1.3 Where the Two Specializations Meet: EDR Systems 7
2. The Scientific Problem 8
3. Project Objective 9
3.1 Summary of the Reference Paper 9
3.2 Analytical Notes on the Reference Paper 10
3.3 What We Aim to Achieve 11
4. Related Work 13
4.1 Selection Methodology 13
4.2 Previous Studies 13
4.3 Summary Table of Previous Studies 19
4.4 Synthesis and Positioning of the Project 20
5. Proposed Solution 21
5.1 The Idea 21
5.2 The Experiment of the Current Phase 22
5.3 How the Solution Will Be Evaluated 23
6. Practical Applications 24
7. Scientific Challenges 25
Chapter 1: Theoretical Background 26
Section 1: Endpoint Security and EDR Systems 26
Section 2: Machine Learning and Deep Learning for Classification 31
Section 3: Natural Language Processing and Transformers 36
Summary of Chapter 1 38
References 40

List of Tables

Table (1): Algorithms and their settings in the reference paper [1] 10
Table (2): Results of the reference paper [1] 10
Table (3): Project phases 11
Table (4): Summary of previous studies 19
Table (5): What the studies show per axis, and the impact on the project 20
Table (6): Endpoint data sources and their use in detection 27
Table (7): Comparison of detection methods 29
Table (8): Comparison of the two datasets 30
Table (9): The learning methods compared in the project 35
Table (10): BERT-family models relevant to the project 37

List of Figures

Figure (1): Block diagram of the proposed intelligent EDR system and the phase that builds each block 21
Figure (2): Flowchart of the experiment in the current phase 22

List of Abbreviations

Abbreviation	Meaning
AST	Abstract Syntax Tree
AUC	Area Under the ROC Curve
AWS	Amazon Web Services
BERT	Bidirectional Encoder Representations from Transformers
BPE	Byte-Pair Encoding
C2	Command and Control
CIC	Canadian Institute for Cybersecurity
CNN	Convolutional Neural Network
DL	Deep Learning
DNS	Domain Name System
EDR	Endpoint Detection and Response
EPP	Endpoint Protection Platform
FPR	False Positive Rate
GBM	Gradient Boosting Machines
LLM	Large Language Model
LotL	Living off the Land
LSTM	Long Short-Term Memory
ML	Machine Learning
MLM	Masked Language Modeling
MLP	Multilayer Perceptron
NLP	Natural Language Processing
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
SMOTE	Synthetic Minority Over-sampling Technique
SOC	Security Operations Center

List of Symbols

Symbol	Meaning
TP	True Positives: attack samples classified as attacks
TN	True Negatives: benign samples classified as benign
FP	False Positives: benign samples classified as attacks
FN	False Negatives: attack samples classified as benign
p	Predicted probability that a sample is an attack
x', x_{min}, x_{max}	Scaled value, minimum and maximum of a feature
G, p_k	Gini impurity and proportion of class k in a node
s(x, n)	Isolation Forest anomaly score
v	Learning rate of gradient boosting
w_i, b	Weights and bias of a neuron
L	Loss function (binary cross-entropy)
c_t, f_t, i_t	LSTM cell state, forget gate and input gate
Q, K, V, d_k	Attention queries, keys, values and key dimension

1. Preface

1.1 Importance of the General Specialization: Artificial Intelligence Engineering

Artificial intelligence has become the main tool for extracting patterns from large data and making automated decisions. Health, finance, industry and telecommunications all depend on it today, because the volume of data has outgrown manual analysis. The value of AI engineering lies in turning theoretical models into systems that work in real environments and cope with their constraints of time, memory and incomplete data.

1.2 Importance of the Specific Specialization: Systems and Information Security

Systems and information security protects the confidentiality, integrity and availability of data. Its importance grows as organizations move to digital work and their attack surface expands to every computer, phone and server connected to the network. These devices, called endpoints, are the attacker's first target, because they are where commands are executed and data is stored.

1.3 Where the Two Specializations Meet: EDR Systems

The two specializations meet in Endpoint Detection and Response (EDR) systems. These systems monitor processes, commands, files and network connections on every device, collect them in a central database, and analyze them to detect malicious behavior [1]. They provide four core functions: continuous monitoring; automated response, such as isolating an infected device and stopping a malicious process; incident response support; and proactive threat hunting [1].

This monitoring produces volumes of logs that cannot be triaged with manual rules alone. This is where machine learning and natural language processing come in: the first learns patterns from data, and the second handles the textual part of it, namely commands and log messages.

2. The Scientific Problem

Traditional EDR systems rely on signatures and fixed rules, which leads to three related problems:

1. **Weak detection of new and obfuscated attacks.** A signature matches a known pattern, so an unseen attack slips past it. So does an attack that uses the system's own legitimate tools (Living off the Land) such as PowerShell, or one that obfuscates the syntax of a command without changing its function.
2. **High false alarm rates.** Generic rules classify legitimate administrative activity as attacks, so the security analyst is flooded with alerts and the response to real threats is delayed. The reference paper reports a false positive rate of 5.6% for the traditional system [1].
3. **Ignoring the textual meaning of endpoint data.** Many studies, including the reference paper, represent activity as statistical figures about network flows (flow duration, number and size of packets). Command lines and log messages, however, are texts with vocabulary, order and context, and turning them into numeric columns loses this meaning.

The project's problem therefore is: **can deep learning and natural language processing models, especially Transformer models such as BERT, classify endpoint logs and commands with higher accuracy and fewer false alarms than the Random Forest algorithm?** Random Forest was chosen for comparison because the reference paper showed it to be the best among traditional machine learning algorithms.

This leads to the question of the current phase (the semester project): **when classifying the same data, what is the actual difference between Random Forest and deep learning models in accuracy, false positive rate and execution time?**

3. Project Objective

3.1 Summary of the Reference Paper

The project builds on the paper by Abu Lail, Shkoukani and Abusaimeh published in 2026, entitled «Endpoint Detection And Response Systems (EDR) Enhancing Using Classification Techniques: A Machine Learning Approach» [1].

Aim and idea. The paper aims to improve the performance of traditional EDR systems by combining them with machine learning classification algorithms. It addresses three weaknesses of the traditional system: high false alarms, weak anomaly detection, and poor use of historical and real-time data. To demonstrate the benefit, the authors compared three algorithms against a traditional signature- and rule-based EDR system.

Dataset. The authors used a dataset they named CSE-CIC-IDS2018, stating that the Canadian Institute for Cybersecurity (CIC) created it in 2018 on AWS. They described it as containing 2,830,540 labeled records, 83 features and 15 classes: a benign class making up 83.34% of the records and 14 attack types, including denial of service, port scanning, web attacks and botnets. The features describe network flows: flow duration, number and size of packets, protocol flags, and byte and packet rates per second.

Preprocessing. Processing was done in Python on Jupyter Notebook and Google Colab, and included the following steps:

- removing non-numeric features and duplicate records;
- removing rows containing empty or infinite values, and rows that repeat the column names;
- removing columns whose values are all zero;
- scaling numeric values to the range 0–1 (Min-Max scaling);
- adding a binary label: 1 for attack and 0 for benign;
- splitting the data into 70% for training and 30% for testing.

67 features remained after preprocessing.

Algorithms and settings. The authors chose three algorithms covering supervised and unsupervised learning, and tuned their parameters with 5-fold cross-validation. Table (1) shows the settings reported in the paper.

Table (1): Algorithms and their settings in the reference paper [1]

Algorithm	Learning type	Idea	Settings
Random Forest	Super-vised	Many decision trees, each learning from a ran-dom subset of the data; the decision is a ma-jority vote	100 trees, max depth 10
Isolation Forest	Unsuper-vised	Isolates points with random splits; a point isolated with few splits is anomalous	100 trees, assumed contamination 0.01
Gradient Boosting Machines (GBM)	Super-vised	Trees built sequentially, each correcting the errors of the previous one	100 trees, learning rate 0.1

Results. Results were measured with five metrics: Accuracy, Precision, Recall, F1-score and False Positive Rate (FPR).

Table (2): Results of the reference paper [1]

Algorithm	Accuracy	Precision	Recall	F1-Score	FPR
Random Forest	98.5%	97.8%	98.2%	97.8%	2.1%
Isolation Forest	95.2%	94.5%	95.0%	94.7%	1.5%
GBM	97.2%	96.5%	97.0%	96.7%	1.8%
Traditional EDR	89.3%	88.0%	89.0%	88.5%	5.6%

The paper concluded that Random Forest is the best at detecting malicious activity, that Isolation Forest has the fewest false alarms, and that GBM lies between them. It recommended extending the framework to other domains and improving accuracy and computation speed.

3.2 Analytical Notes on the Reference Paper

Studying the paper led to the following notes, which shape the way we build our project:

1. **Nature of the data.** The data used consists of network flows represented by numeric features. It does not contain what characterizes an endpoint: command lines, processes and log messages. This gap is the starting point of our project.
2. **Dataset description.** The reported characteristics (2,830,540 records and 15 classes with names such as DoS Hulk, PortScan and Heartbleed) are very close to the CIC-IDS2017 dataset, which contains 2,830,743 records. CSE-CIC-IDS2018, by contrast, contains about 16.2 million records [12]. We will confirm the dataset to adopt with the supervisor before rebuilding the experiment.
3. **The traditional baseline.** The paper does not explain how the traditional EDR system was applied to flow data, so it cannot be rebuilt as described.
4. **Limits of the comparison.** The paper is limited to traditional machine learning algorithms and to binary classification. It did not test deep learning, and it did not process textual data.

5. **The F1 formula.** The formula appears in the paper with the sum in the numerator and the product in the denominator. Our project adopts the standard formula given in Section 3.3.
6. **Internal consistency of the results.** GBM's accuracy is 97.2% in the table but 96.7% in the text. From the reported Precision, Recall and FPR of Random Forest, its accuracy should fall between about 97.9% and 98.2%, not 98.5%. And an Isolation Forest set to a contamination of 0.01 cannot flag 95% of the attacks when attacks make up around 17% to 20% of the data. These points make an independent rebuild of the experiment necessary.

3.3 What We Aim to Achieve

Our project is entitled «Enhancing EDR using NLP and Transformers». We start from the result of the reference paper and add what it did not address: deep learning first, then processing the text of commands and logs with BERT models. Two works from 2024 paved the way for this direction: Portnoy et al. used large language models to detect hands-on-keyboard attacks within EDR solutions [14], and Sharif et al. pre-trained a language model on sequences of EDR events to produce representations of processes [15].

The project is carried out in three phases, shown in Table (3).

Table (3): Project phases

Phase	Work	Expected output
Semester project (current phase)	Study and comparison of Random Forest and deep learning models in classifying EDR records	Theoretical study and an experimental comparison with unified metrics
Graduation project 1	Building and training a BERT model to detect suspicious commands and extract features	A model trained and evaluated on textual data
Graduation project 2	Integrating the system with a live environment and developing an automated threat response mechanism	An experimental EDR system working in a live environment

In the current phase we aim at four things:

1. **Rebuilding the baseline:** training Random Forest with the reference paper's settings (100 trees, maximum depth 10) and verifying its results.
2. **Training deep learning models** on the same data with the same split: a Multilayer Perceptron (MLP), a Convolutional Neural Network (CNN) and an LSTM network.
3. **Comparing them with the five metrics** used in the reference paper, adding training time and prediction time.
4. **Identifying a source of textual data** (endpoint commands and logs) suitable for the BERT phase.

The comparison metrics are computed from the elements of the confusion matrix using equations (1) to (5):

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (1)$$

$$Precision = TP / (TP + FP) \quad (2)$$

$$Recall = TP / (TP + FN) \quad (3)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \quad (4)$$

$$FPR = FP / (FP + TN) \quad (5)$$

where TP is the number of attack samples classified as attacks, TN the number of benign samples classified as benign, FP the number of benign samples classified as attacks, and FN the number of attack samples classified as benign.

4. Related Work

4.1 Selection Methodology

The studies were selected according to four criteria:

- published in 2025 or 2026, excluding the reference paper [1];
- directly related to one of the project's four axes: language models on endpoint data; deep learning on device logs; language models in the security analyst's work; and the comparison between machine learning and deep learning;
- presenting experimental results or a methodological analysis useful for designing our experiments;
- published in a recognized conference or journal. One paper available only as an arXiv preprint was kept because of its close relevance to graduation project 1, and this is noted where it is presented.

4.2 Previous Studies

4.2.1 HyperGLLM: Hypergraph-Enhanced Large Language Models for Endpoint Threat Detection [2]

Zhou et al. published this study in the proceedings of AAAI-26. It addresses threat detection in EDR logs with large language models (LLMs). It argues that these models can analyze behavior in logs but are hindered by two things: the large volume of events, and the interleaving of different behavior sequences within one log. This makes long-context processing and the detection of stealthy threats difficult.

The proposed framework works in three steps. It first builds an attribute–value level graph that compresses textual redundancy in events. It then adds a hypergraph module with multi-level clustering that captures behavioral correlations among interleaved events. Finally, it aligns these representations with a language model that infers malicious behavior. The authors built the EDR3.6B-63F dataset of 3.6 billion events from 63 behavioral families. The framework reduced the false positive rate to 1.67% and achieved 94.65% accuracy across the 63 families.

Relation to the project: it is the closest to our final goal, analyzing EDR logs with a language model to reduce false alarms. It also warns us that feeding the full raw log into the model is impractical, and that the BERT phase needs events to be compressed and organized before classification.

4.2.2 Command-Line Obfuscation Detection in Real-World Telemetry under Extreme Class Imbalance [3]

Outrata et al. published this study at NLPAICS 2026. Its subject is command-line obfuscation: changing the syntax of a command while preserving its function in order to evade endpoint protection tools. The study proposes a detection method specific to command-line data, built on a small Transformer model trained for this task and optimized for fast inference over massive data streams.

The evaluation had two phases. The first used a controlled dataset simulating realistic command-line telemetry with extreme class imbalance, where the model outperformed previous approaches. The second used several days of high-volume real data from diverse environments. The results showed high precision and computational efficiency that allow processing command logs at scale while reducing analyst workload.

Relation to the project: it supports the hypothesis our project rests on: the command line resembles a language whose structure a Transformer can learn. It shows that a small model is enough, which suits our computing resources, and confirms that class imbalance is a core challenge that must enter the design of our experiments.

4.2.3 Semantic Clustering of Obfuscated Command-Line Detections for Alert Reduction [4]

Štěpánková et al. published this study at NLPAICS 2026 as a continuation of the previous study by the same team. It addresses what happens after detection: the large number of alerts produced by an obfuscation detector. It proposes a post-processing step that groups alerts into semantically coherent clusters, so that the analyst handles one alert per cluster.

Commands are first encoded with Sentence-BERT embeddings and then grouped by hierarchical agglomerative clustering. A large language model then evaluates representative samples of each cluster to discard false alarms. The method was evaluated on real data from a commercial endpoint protection system. On one week of data, alert volume fell by about 98% while clusters remained pure and semantically coherent.

Relation to the project: it serves directly the goal of reducing false alarms stated in our project description. It suggests a two-stage design (a detector, then semantic grouping of alerts) that can be adopted in graduation project 2, and shows that BERT representations are useful for grouping, not only for classification.

4.2.4 Detecting Fileless Cryptojacking in PowerShell Using AST-Enhanced CodeBERT Models [5]

Varlioglu et al. published this paper in 2026 as an arXiv preprint. It studies fileless cryptojacking attacks executed through PowerShell on Windows. These attacks are loaded into memory

and so evade file-based detection, and their processes may remain active on the device even after the malicious script is deleted.

The authors collected a dataset of PowerShell scripts for this type of attack, represented the scripts in a way that integrates their Abstract Syntax Tree (AST), and then tested three models: LSTM, BiLSTM and CodeBERT. The CodeBERT model fine-tuned on the AST representation achieved a high recall, and the authors concluded that integrating the syntax tree with models pre-trained on programming languages matters.

Relation to the project: it is the closest to graduation project 1, fine-tuning a BERT-family model on endpoint commands and scripts. It compares recurrent networks with Transformers on textual data, the same comparison we will run later. We take its results with caution because it has not yet been peer-reviewed.

4.2.5 iCNN-LSTM+: Batch-Based Incremental Ransomware Detection Using Sysmon [6]

Ispahany et al. published this study in IEEE Access in 2025. It presents iCNN-LSTM+, a system for real-time ransomware detection on Windows devices. Its data source is Sysmon logs, which record system events on the endpoint, and the model combines a CNN with an LSTM network.

The main addition is batch-based incremental learning: the model adapts to unseen ransomware samples without full retraining, overcoming the limits of static models. The system achieved an average F2-score of 99.65%, with a false positive rate of 0.16% and a false negative rate of 3.96%, on highly imbalanced data. The authors proposed a new metric (Weighted Efficiency Score) that balances detection accuracy and throughput, and used parallel LSTM processing with an attention mechanism to raise throughput.

Relation to the project: it is a direct example of our current phase's topic: deep learning classifying real endpoint logs. It points us to Sysmon as a source for collecting our own logs, and raises the issue of attacks changing over time, which becomes essential when moving to a live environment in graduation project 2.

4.2.6 «Sometimes Simpler is Better»: An Analysis of Provenance-Based Intrusion Detection Systems [7]

Bilot et al. published this study at USENIX Security 2025. It reviews intrusion detection systems based on provenance graphs. These graphs represent system entities (processes, files, connections) as nodes and their interactions as edges; the system then learns normal behavior and treats deviations as anomalies.

The authors re-implemented eight recent systems in a unified framework. Although the original studies reported near-perfect detection, the authors identified nine shortcomings that prevent

practical use and measured their impact with metrics tied to operational security work. They then showed that most systems add unnecessary complexity: a simple neural network achieved the best performance on most of the tested DARPA datasets, while being lighter, faster and real-time.

Relation to the project: it is a methodological lesson for our comparison. The more complex model is not necessarily better, and a fair comparison requires a unified framework with the same data and the same split for all models. We therefore keep Random Forest and a simple network as baselines before moving to BERT, and it warns us against relying on high accuracy figures without checking practicality.

4.2.7 Language Models in Threat Hunting: Detecting Living off the Land Techniques [8]

Konstantinou et al. published this study in 2025 in Machine Learning and Knowledge Extraction. It investigates the use of general-purpose language models (ChatGPT, Google Gemini and Microsoft Copilot) in threat hunting, specifically for detecting Living off the Land techniques, in which the attacker uses legitimate system tools so that his activity blends into normal activity and is hard to detect automatically. The question is whether these models produce effective queries for security tools that help organizations with limited budgets and expertise.

The authors built a test environment simulating these techniques and used the queries generated by the models to detect malicious activity. The results showed that the models do not consistently produce accurate and reliable queries, especially with users of varying expertise. The authors concluded that they are useful as an assisting tool within a broader security strategy, not as a standalone detection tool.

Relation to the project: it supports our methodological choice. Relying on a general chat model through questions is not enough for detection; a specialized model fine-tuned on labeled data, such as BERT, is more suitable. It also identifies the hardest class of attacks that our data must include.

4.2.8 Deep Learning vs. Machine Learning for Intrusion Detection: A Comparative Study [9]

Ali et al. published this study in 2025 in Applied Sciences. It compares deep learning models with traditional machine learning algorithms for intrusion detection on the CICIDS2017 dataset (about 2.83 million records and 79 columns). The deep models were MLP, CNN and LSTM; the traditional algorithms were logistic regression, naive Bayes, Random Forest, k-nearest neighbors and decision tree.

The authors handled class imbalance with SMOTE, removed highly correlated features with a 0.85 threshold, and tuned parameters by random search. Random Forest achieved the highest

accuracy (99.88%), followed by the decision tree (99.83%). The deep models reached 97% for MLP and 98% for both CNN and LSTM. The study discussed the higher computational cost of deep models and the difficulty of running them in real time.

Relation to the project: its experiment is very close to that of our current phase, and its result is a reference point for comparison. Random Forest outperforming on tabular data supports our hypothesis that the advantage of deep learning appears in textual and sequential data, not in ready-made statistical features. We take from it its preprocessing steps (SMOTE and removal of correlated features) and per-class evaluation.

4.2.9 Evaluating Large Language Models for Intrusion Detection Compared with ML and DL Baselines [10]

Mehavilla et al. published this study in Artificial Intelligence Review (vol. 59, 2026). It offers a systematic comparison that tests large language models as standalone intrusion detection classifiers. Three models (GPT-2, GPT-Neo-125M and LLaMA-3.2-1B) were tested in binary and multi-class classification on Zeek logs derived from the CIC IoT 2023 dataset.

These models were compared with machine learning algorithms (XGBoost, Random Forest and decision tree) and deep learning models (MLP, GRU and LeNet-5). The comparison covered not only detection effectiveness but also inference speed and resource consumption. The language models exceeded 95% F1 but did not outperform the best machine learning algorithms: XGBoost achieved a higher F1 (96.96%) while using only 4% of the available processor capacity.

Relation to the project: it shows that language models do not beat ensemble trees when the data is structured flows. This directs our project: we apply BERT where the data is actual text (commands and logs), not to numeric features. We also add inference time and resource consumption to our comparison.

4.2.10 Ensemble vs. Hybrid Deep Learning Models on CSE-CIC-IDS2018 [11]

Dharel et al. published this study in 2026 in the International Journal on Engineering Technology. It compares three intrusion detection models on CSE-CIC-IDS2018: Random Forest, XGBoost and a hybrid CNN-LSTM model. Preprocessing included data cleaning, feature encoding, scaling and feature selection, and results were measured with accuracy, precision, recall, F1, false positive rate and the ROC curve.

XGBoost performed best, with 97.97% accuracy, an F1 of 0.9797 and a false positive rate of 0.0051. Random Forest followed with 97.90% accuracy, then CNN-LSTM with 97.40%. The authors concluded that ensemble methods suit tabular data, hybrid deep models suit sequential data, and model choice must balance accuracy, speed and deployability.

Relation to the project: it uses the dataset named in the reference paper and the same base algorithm, so its result (97.90% for Random Forest) gives us an expected range when rebuilding the baseline, alongside the 98.5% of the reference paper [1]. It is the study closest to our semester comparison.

4.3 Summary Table of Previous Studies

Table (4): Summary of previous studies

#	Title	Authors	Venue	Year	Relation to the project
1	HyperGLLM: An Efficient Framework for Endpoint Threat Detection via Hypergraph-Enhanced Large Language Models	Zhou, Pan, Peng, Huang, Zheng	Proc. AAAI-26	2026	Language model on EDR logs to reduce false alarms
2	Command-Line Obfuscation Detection in Real-World Telemetry under Extreme Class Imbalance	Outrata, Štěpánková, Polák, Kopp	Proc. NLPAICS	2026	Small Transformer classifying command lines
3	Semantic Clustering of Obfuscated Command-Line Detections for Alert Reduction	Štěpánková, Outrata, Kopp	Proc. NLPAICS	2026	Sentence-BERT to cut alert volume
4	Detecting Fileless Cryptojacking in PowerShell Using AST-Enhanced CodeBERT Models	Varlioglu, Elsayed, Ozer, ElSayed, Emmert	arXiv preprint	2026	Fine-tuning CodeBERT on commands (closest to phase 2)
5	iCNN-LSTM+: A Batch-Based Incremental Ransomware Detection System Using Sysmon	Ispahany, Islam, Khan, Islam	IEEE Access	2025	Deep learning on Sysmon endpoint logs
6	Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based IDS	Bilot et al.	USENIX Security	2025	Simple models can match complex ones; fair comparison
7	Leveraging LLMs for Non-Security Experts in Threat Hunting: Detecting Living off the Land Techniques	Konstantinou et al.	MAKE (MDPI)	2025	General LLMs are unreliable alone; need a fine-tuned model
8	Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study	Ali, Thakur, Schmeelk, Debello, Dragos	Applied Sciences (MDPI)	2025	The same RF vs. DL comparison on CIC-IDS2017
9	Evaluating LLMs Effectiveness for Flow-Based Intrusion Detection: A Comparative Study with ML and DL Baselines	Mehavilla, Rodríguez, García, Alesanco	Artificial Intelligence Review (Springer)	2026	LLMs do not beat trees on structured flows
10	Comparative Analysis of Ensemble ML and Hybrid DL Models for Intrusion Detection	Dharel, K.C., Maharjan	Int. J. on Engineering Technology	2026	The same comparison on CSE-CIC-IDS2018

4.4 Synthesis and Positioning of the Project

Table (5): What the studies show per axis, and the impact on the project

Axis	What the studies show	Impact on our project
Language models on endpoint data (4.2.1–4.2.4)	Transformers succeed when the data is text: commands, scripts and EDR logs. A small specialized model is enough, and long logs need compression	We adopt BERT for the textual phase, and plan to compress events before classification
Deep learning on device logs (4.2.5–4.2.6)	Deep learning is effective on Sysmon logs, and the simplest model may match the most complex	We keep simple baselines and compare within one unified framework
Language models in the analyst's work (4.2.7)	General chat models used through questions are unreliable for detection	We fine-tune a specialized model on labeled data
ML vs. DL comparison (4.2.8–4.2.10)	On tabular data, ensemble trees match or beat deep models	We expect a similar result in the semester comparison, and expect deep models to show their advantage on text

None of the ten studies compares, within one framework, Random Forest, deep learning and a BERT-family model on endpoint data. This is what our project aims at through its three phases: a comparison on numeric data first, then a move to text, then operation in a live environment.

5. Proposed Solution

5.1 The Idea

The project treats what an EDR agent records on the endpoint as data that can be learned from in two complementary forms: **numeric features** that summarize activity (such as flow statistics), and **text**, namely the command lines and log messages that describe exactly what was executed. The solution replaces the signature-and-rule analysis engine of a traditional EDR system with a learning-based detection engine, and then adds the steps that turn a model's output into a usable alert: a tunable decision threshold, grouping of similar alerts, and automated response.

The detection engine is built in two stages. In the current phase it works on numeric flow features, where Random Forest serves as the baseline and is compared with deep learning models under identical conditions. In graduation project 1 a second detector is added that reads command-line text with a fine-tuned BERT model. Graduation project 2 connects the whole chain to a live lab environment. Figure (1) shows the overall architecture and which phase builds each block.

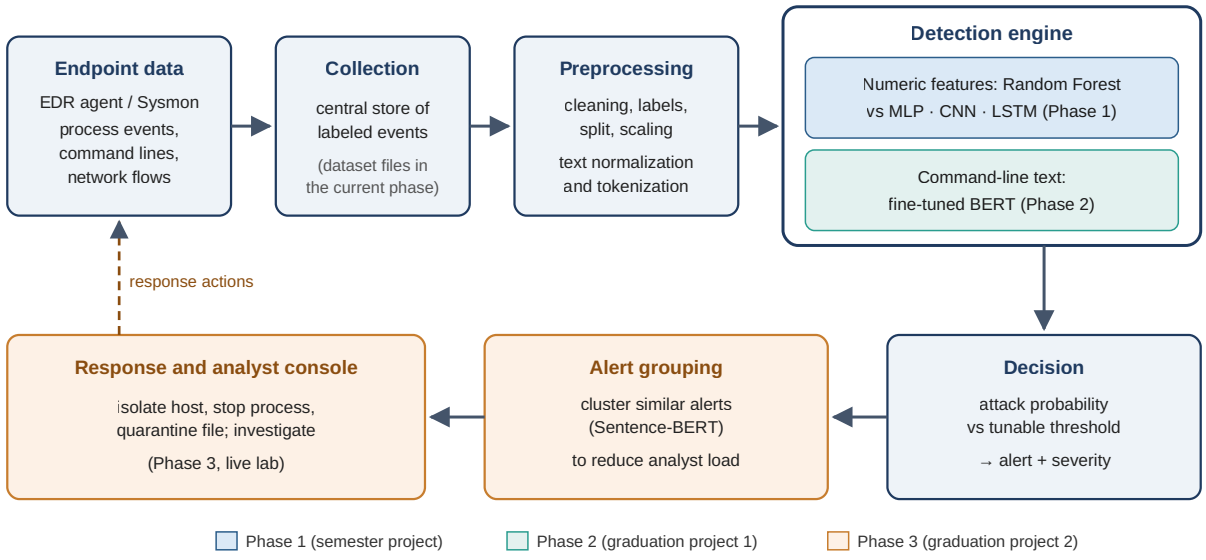


Figure (1): Block diagram of the proposed intelligent EDR system and the phase that builds each block

The blocks work as follows:

1. **Endpoint data.** The agent records process creation, command lines, file and registry changes, and network connections (Section 1.3 of Chapter 1). In the current phase this block is represented by the labeled flow records of CIC-IDS2017 [13].

2. **Collection.** Events from all endpoints are stored centrally with their labels, so that models can be trained and evaluated.
3. **Preprocessing.** Numeric data is cleaned, labeled as benign or attack, split and scaled; text is normalized and split into tokens for the language model.
4. **Detection engine.** A classifier outputs, for each event, the probability that it is malicious. The numeric detector is the subject of the current phase; the text detector is added in graduation project 1.
5. **Decision.** The probability is compared with a threshold. Raising the threshold reduces false alarms at the cost of missed attacks, so its value is chosen from the measured trade-off rather than fixed in advance.
6. **Alert grouping.** Similar alerts are clustered into one case, following the approach of Štěpánková et al. [4], to reduce alert fatigue.
7. **Response and analyst console.** Confirmed alerts trigger automated actions on the endpoint and are presented to the analyst for investigation.

5.2 The Experiment of the Current Phase

The semester project implements and evaluates the first four blocks on numeric data. Its procedure, shown in Figure (2), rebuilds the reference paper's pipeline [1] and extends it with deep learning models, so that every model sees exactly the same data, the same split and the same metrics.

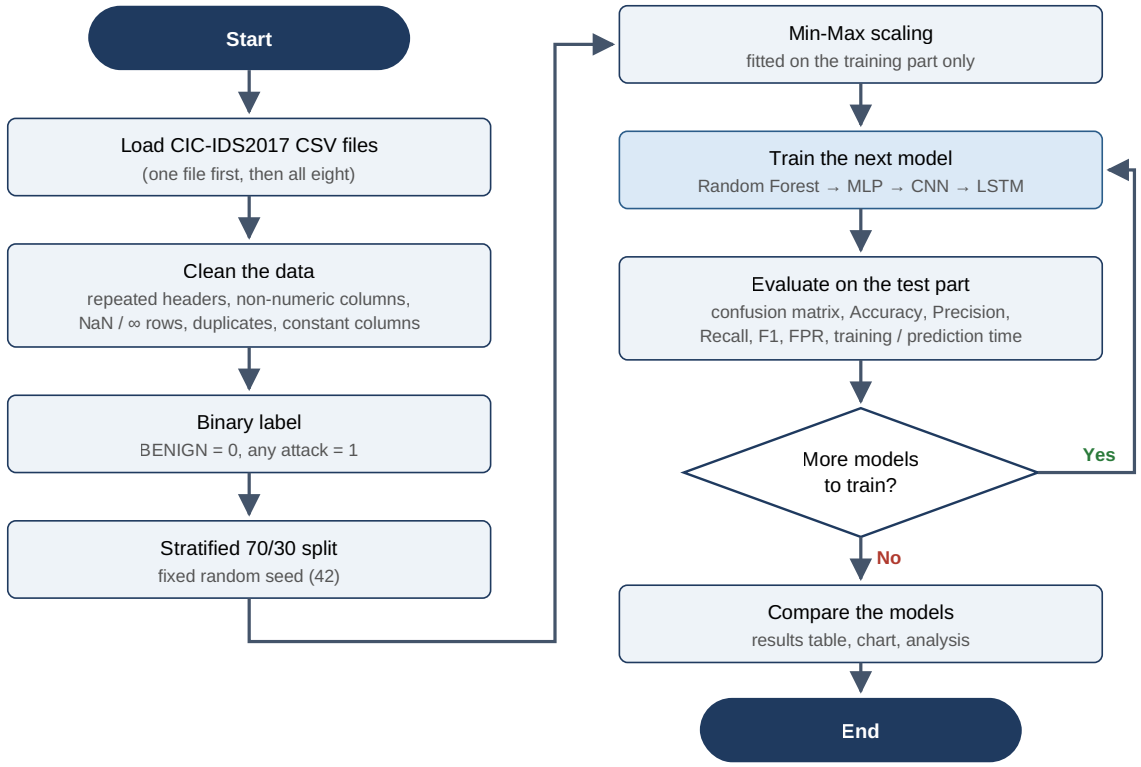


Figure (2): Flowchart of the experiment in the current phase

The design choices follow the reference paper wherever possible, so that the baseline can be checked against its reported results: the same cleaning steps, Min-Max scaling, binary labels, a 70/30 split, and Random Forest with 100 trees and a maximum depth of 10 [1]. Two choices go further. The split is stratified with a fixed seed so that the experiment is reproducible, and the scaler is fitted on the training part only, which prevents information from the test set leaking into training (Section 2.2 of Chapter 1). The deep learning models are deliberately small, and all of them use early stopping and the same decision threshold of 0.5.

The pipeline has been implemented in Python with scikit-learn and TensorFlow/Keras as four scripts (data preparation, training, reporting and a live alert demo) and has been run end to end on the first CIC-IDS2017 file. The full results and their analysis belong to the results chapter of the report.

5.3 How the Solution Will Be Evaluated

Each phase is judged against the previous one with the five metrics of Section 3.3 plus training and prediction time:

- **Phase 1:** deep learning models against the Random Forest baseline, on the same numeric data.
- **Phase 2:** the BERT text detector against a baseline trained on simple text features (such as token frequencies) and against the Phase 1 results, with particular attention to false positive rate and to obfuscated commands.
- **Phase 3:** detection delay, false alarms per day and alert volume after grouping, measured in the live lab.

6. Practical Applications

The project serves any organization that runs networked devices and needs early intrusion detection. Its main applications are:

- **Security Operations Centers (SOC):** automatic triage of alerts ranked by severity, so analysts focus on real cases.
- **Small and medium enterprises:** a detection system built on open-source tools, as a low-cost alternative to commercial EDR solutions.
- **Detecting fileless and Living off the Land attacks:** analyzing the text of commands and scripts reveals what file-signature matching misses.
- **Automated incident response:** isolating the infected device or stopping the malicious process upon detection, the goal of graduation project 2.
- **Digital forensics:** classifying device logs after an incident to reconstruct the attack sequence quickly.
- **Education and research:** a unified comparison platform for machine learning, deep learning and Transformer algorithms on security data.

7. Scientific Challenges

1. **Scarcity of labeled EDR data.** Real endpoint logs belong to companies and contain sensitive information, and public datasets of them are few. A suitable textual dataset must be found or generated in a lab environment.
2. **The gap between the reference paper's data and endpoint data.** CSE-CIC-IDS2018 consists of network flows represented by numeric features, not textual commands or logs. The semester comparison is possible on it, but the BERT phase needs another data source.
3. **Verifying the reference paper's dataset.** Its description (about 2.83 million records and 15 classes) matches CIC-IDS2017 [9, 13], whereas CSE-CIC-IDS2018 contains about 16.2 million records [12]. The dataset to adopt must be settled before rebuilding the baseline.
4. **Defining the traditional baseline.** The reference paper does not explain how the traditional EDR system was applied to flow data. We need a clear and reproducible definition of this baseline.
5. **Class imbalance.** Attacks are few compared with benign activity, so overall accuracy becomes misleading. We therefore rely on precision, recall and false positive rate, and evaluate each class separately.
6. **Representing commands and logs as text.** Commands are not natural language: they contain paths, symbols, Base64 encoding and deliberate obfuscation. BERT models have a maximum input length, so a suitable segmentation and compression method must be chosen.
7. **Computational cost and real-time operation.** Deep models and Transformers are heavier than Random Forest to train and to run, and our computing resources are limited.
8. **Attacks changing over time.** A model trained on old attacks weakens against new ones, so generalization to attacks absent from training must be tested.
9. **Interpretability.** The security analyst needs to know why an alert was raised, and deep models are less transparent than trees.

Chapter 1: Theoretical Background

This chapter presents the theoretical foundations of the project in three sections: endpoint security and EDR systems; machine learning and deep learning in classification; and natural language processing and Transformers. Together they cover the ideas on which the three phases of the project rest and which the team needs throughout the work.

Section 1: Endpoint Security and EDR Systems

1.1 The Endpoint and the Attack Surface

An endpoint is any device that connects to the organization's network and runs software: an employee's laptop or desktop, a server, a smartphone, or a virtual machine in the cloud. What distinguishes it from network devices (routers and firewalls) is that it has a full operating system on which the user, or the attacker, can run whatever they want.

The attack surface is the set of points through which an attacker can get in. This surface has widened over the last decade for three reasons: the spread of remote work, which took devices outside the protection of the corporate network; the diversity of operating systems and personal devices used for work; and the adoption of cloud services, which made the traditional network perimeter less meaningful.

The endpoint has therefore become the attacker's first target. It is where a malicious email attachment is opened, where commands are typed, and where the credentials that allow moving to other devices are stored. Most major attacks start from a single compromised device and then spread.

1.2 From Antivirus to EDR

Traditional protection. The first protection relied on antivirus software: a program that scans files and compares them with a database of signatures of known malware, blocking whatever matches. Other layers were later added in what is called an Endpoint Protection Platform (EPP): a host firewall, control of external devices, and disk encryption. What these layers share is that they are preventive: they block the known before it runs, and they do not answer the question «what happened on the device after the attacker got past prevention?».

The emergence of EDR. The term Endpoint Detection and Response was coined in 2013 by Anton Chuvakin of Gartner to describe a new category of tools that assume a breach will happen and focus on seeing it and handling it quickly. An EDR system continuously collects in-

formation about the activity of each device, stores it centrally, analyzes it to detect malicious behavior, and enables a response [1].

Components of an EDR system. A typical EDR system has four parts:

1. **The agent:** a lightweight program installed on each endpoint. It monitors processes, files, the network and the registry, sends events to the server, and executes response commands.
2. **The central server or data lake:** receives events from thousands of devices and stores them for a period that allows retrospective investigation.
3. **The analysis engine:** applies rules, signatures, behavioral models and machine learning models to the events to produce alerts.
4. **The analyst console:** an interface where the security analyst sees alerts, traces the process tree, and launches response actions.

The four functions of EDR. The reference paper summarizes EDR's functions as four [1]: continuous monitoring of device behavior; automated response, such as isolating the infected device from the network, stopping the malicious process and quarantining the file; incident response support with ready investigation workflows; and proactive threat hunting integrated with threat intelligence. The essential difference from antivirus is that EDR records everything and then judges, while antivirus judges only at the moment a file is scanned.

1.3 Endpoint Data Sources

The value of an EDR system is the value of the data it collects. Table (6) shows the main sources and what each reveals.

Table (6): Endpoint data sources and their use in detection

Source	Example event	What it reveals
Process creation	powershell.exe launched by winword.exe	Suspicious execution chains: a document running commands
Command line	The full command with its arguments	The attacker's intent: downloading, encrypting, disabling protection
File system	An executable created in a temp folder; thousands of files modified quickly	Malware dropping; ransomware
Registry	A new autorun key	Persistence after reboot
Network connections	An unusual process connecting to an external address	Command and control (C2) and data exfiltration
DNS queries	Random-looking domain names	Covert communication channels
Library loading	A library loaded from a non-standard path	Code injection and DLL hijacking
Logon events	An administrative logon at an unusual time	Credential theft and lateral movement

Windows logs and Sysmon. Windows provides built-in logs (Windows Event Log) for security, system and applications. The free Sysmon tool from Microsoft's Sysinternals suite extends them by recording detailed events with fixed identifiers: process creation (ID 1) with the full command line, file hash and parent process; network connection (3); image loaded (7); file created (11); registry value set (13); and DNS query (22). Sysmon logs are the most widely used source in academic research that simulates EDR data, because they are free, detailed and can be collected in a small lab, as in the ransomware detection study by Ispahany et al. [6].

Linux systems. On Linux, the auditd system records system calls and program execution, and modern EDR tools rely on eBPF to monitor the kernel at low cost. The idea stays the same: recording who executed what, when, and with which arguments.

1.4 Detection Methods

Signatures and rules. A signature is a fixed pattern that identifies something known to be malicious: a file hash, a byte sequence, or a regular expression on a command line. Rules generalize this idea into logical conditions on events, such as the open Sigma rules, which describe attack patterns in logs in a unified format that can be translated to any platform. The advantages of this method are high precision, speed and ease of interpretation: the analyst knows exactly why the alert was raised. Its drawback is that it sees only what was written for it in advance, so a small change in syntax is enough to evade it.

Behavioral detection. Instead of looking for a fixed pattern, behavioral detection monitors sequences of actions and compares them with documented attacker tactics in the MITRE ATT&CK framework, which classifies real attack techniques into stages: initial access, execution, persistence, privilege escalation, defense evasion, lateral movement and others. For example: opening an Office document results in a PowerShell process that connects to the Internet and writes an executable. Each step alone is normal, but the sequence matches a known attack technique.

Anomaly detection and machine learning. This type learns what is «normal» from past data and treats whatever departs from it as suspicious. This is where machine learning algorithms come in: supervised algorithms that learn from labeled examples (attack or benign), such as Random Forest, and unsupervised algorithms that isolate anomalies without labels, such as Isolation Forest [1]. Their advantage is the ability to detect the unseen; their drawbacks are the need for large data, difficulty of interpretation, and potential false alarms if normal behavior itself changes.

Table (7): Comparison of detection methods

Method	What it detects	Advantage	Limitation
Signatures and rules	The previously known	Precise, fast and interpretable	Evaded by small changes
Behavioral	Known attack sequences	Correlates events and resists simple obfuscation	Requires expertise to write models
Anomaly and machine learning	The unknown and new	Learns from data and generalizes	Needs much data; hard to interpret; false alarms

Modern systems combine the three methods. Our project works on the third layer: improving detection with machine learning and deep learning, and then with language models.

1.5 Attacks That Challenge Traditional Detection

Living off the Land. The attacker does not bring malware but uses tools already present in the system and signed by its vendor: PowerShell, WMI, certutil, rundll32, mshta and others, documented in the open LOLBAS list. Antivirus sees nothing unusual because the tool is legitimate, and the attacker's activity blends with that of the administrator. Detecting these attacks requires looking at how the tool is used, that is, at the command line and its context, which is exactly what Konstantinou et al. study in their attempt to use language models to hunt these techniques [8].

Fileless attacks. They run only in memory and do not write an executable to disk, thereby bypassing file-based scanning. Their processes remain active even after the script that launched them is deleted, as in the PowerShell cryptojacking attacks studied by Varlioglu et al. [5]. The only trace left is the event log: the process that ran and the command it executed.

Command obfuscation. The attacker changes the syntax of a command while keeping its function: Base64-encoding the whole script, splitting strings and reassembling them, replacing argument names with their abbreviations, and inserting meaningless characters. A signature looking for a certain word does not find it, but a model that learns the structure of commands captures the anomaly in the syntax, as Outrata et al. showed with a small Transformer on real data [3].

Ransomware. Ransomware goes through stages: entry, then persistence, then disabling backups and protection tools, then rapid file encryption. The early stages resemble LotL attacks, and the final stage shows clearly in file system logs: thousands of writes and extension changes in a short time. Early detection in the first stages is what prevents loss, and it is the aim of incremental systems such as iCNN-LSTM+ on Sysmon logs [6].

1.6 The Problem of False Alarms

A false positive is an alert about benign activity. It is measured by the false positive rate: the proportion of benign events classified as malicious. The reference paper reports a rate of 5.6% for the traditional system versus 2.1% for Random Forest [1]. The figure may look small, but one endpoint generates thousands of events a day, so 5% of millions of events means thousands of false alerts a day for the security operations center.

The result is a phenomenon known as alert fatigue: the analyst is flooded with alerts, begins closing them without investigation, and the real alert is lost among them. Overall accuracy is therefore not enough to evaluate a detection system; false positive rate and recall must be considered together. Mitigations include raising the decision threshold, ranking alerts by severity, and grouping similar alerts into one, as Štěpánková et al. did to reduce alert volume by about 98% [4].

1.7 Datasets in Intrusion Detection Research

CIC-IDS2017. Produced by the Canadian Institute for Cybersecurity at the University of New Brunswick [13]. It was collected over five working days in July 2017 in a lab network simulating an enterprise network, with benign traffic generated from profiles of real user behavior and attacks executed with known tools: brute force on FTP and SSH, denial of service in its variants, the Heartbleed vulnerability, web attacks (SQL injection, XSS and brute force), infiltration, botnet, distributed denial of service and port scanning. Flow features were extracted from raw traffic with the CICFlowMeter tool, about 80 features per flow, such as flow duration, number of packets in each direction and their sizes, TCP flags, and byte rates per second. The dataset contains 2,830,743 flows [12].

CSE-CIC-IDS2018. Produced by the same institute in cooperation with Canada's Communications Security Establishment on AWS infrastructure, in a much larger environment (dozens of servers and hundreds of devices) over ten days, with similar attack scenarios. It contains about 16.2 million flows with about 80 features [12]. Its large size makes training on it in full heavier, but it is more recent and closer to enterprise networks.

Table (8): Comparison of the two datasets

Property	CIC-IDS2017	CSE-CIC-IDS2018
Producer	CIC, University of New Brunswick	CIC and CSE, on AWS
Collection period	5 days	10 days
Approximate number of flows	2.83 million	16.2 million
Approximate number of features	80	80
Data type	Network flows	Network flows

Network flows versus endpoint data. Both datasets describe network communication from the outside: how many packets passed, of what size, and how fast. They do not contain what an EDR agent sees from inside the device: which process created the connection, with which command, and from which parent process. This difference is what we call in the project the gap between flow data and endpoint data. In the semester project we use flow data because it is labeled and available and allows rebuilding the reference paper's experiment and comparing the algorithms fairly. The language model phase, however, needs textual data from the endpoint itself, such as command lines and Sysmon logs, as discussed in the challenges section.

Summary of Section 1

The endpoint is the first battlefield of an attack, and EDR systems are the tool for visibility and response on it. These systems rely on three detection layers: signatures, behavior, and anomaly detection with machine learning. Modern attacks (living off the land, fileless and obfuscated) bypass the first layer and leave their trace in event logs and command lines, which are text. The problem of false alarms makes their rate a metric no less important than accuracy. On these foundations our project is built: in the current phase we start by comparing algorithms on labeled flow data, in preparation for moving to text.

Section 2: Machine Learning and Deep Learning for Classification

This section presents the learning methods the project compares, the way data is prepared for them, and the problems that affect their evaluation on security data.

2.1 Basic Concepts

Machine learning builds a model from examples instead of from hand-written rules. In **supervised learning** each training example has a known answer (its label), and the model learns to map inputs to labels. In **unsupervised learning** there are no labels; the model looks for structure or for points that do not fit, as anomaly detection does.

The task in the current phase is **binary classification**: each network flow is described by a vector of numeric **features** (flow duration, packet counts and sizes, flag counts, byte rates) and must be classified as benign (0) or attack (1). The model outputs a probability p that the flow is an attack, and a **decision threshold** turns that probability into a class.

A model is useful only if it **generalizes**: it must perform well on data it did not see during training. The data is therefore split into a **training set**, from which the model learns, and a **test set**, used once at the end to measure performance. A model that fits the training data closely but fails on new data is **overfitting**; one too simple to capture the pattern is **underfitting**. Settings chosen before training, such as the number of trees or layers, are **hyperparameters**. They are tuned on the training data, often by **k-fold cross-validation**, which splits the training set into k parts, trains k times leaving one part out each time, and averages the results [23].

2.2 Preparing the Data

Raw intrusion detection data contains duplicates, missing and infinite values, and non-numeric columns, which must be removed or converted before training. Because the features have very different ranges (a duration of microseconds next to a byte rate of millions), they are brought to a common range with **Min-Max scaling**:

$$x' = (x - x_{min}) / (x_{max} - x_{min}) \quad (6)$$

where x_{min} and x_{max} are the minimum and maximum of the feature. Scaling matters most for neural networks and distance-based methods; tree-based models are largely insensitive to it.

Two rules protect the validity of the evaluation. First, the split must be **stratified**, keeping the same proportion of attacks in the training and test sets. Second, every statistic computed from the data, including x_{min} and x_{max}, must be computed from the training set only and then applied to the test set. Computing them from all the data lets information about the test set reach the model, a mistake known as **data leakage**, which makes the reported results look better than they really are.

2.3 Decision Trees and Random Forest

A **decision tree** classifies an example through a sequence of yes/no questions on its features, such as «is the flow duration below 2 seconds?», until it reaches a leaf that gives the class. During training, each question is chosen to make the resulting groups as pure as possible. A common measure of impurity is the **Gini index**:

$$G = 1 - \Sigma_k p_k^2 \quad (7)$$

where p_k is the proportion of examples of class k in a node. G is 0 for a pure node and grows as the classes mix. A single deep tree tends to overfit.

Random Forest, introduced by Breiman [16], reduces this weakness by combining many trees. Each tree is trained on a bootstrap sample (a random sample drawn with replacement) of the training data, and at each split it considers only a random subset of the features. The trees therefore make different errors, and the forest predicts by majority vote; the fraction of trees voting «attack» serves as the attack probability. The main hyperparameters are the number of trees and the maximum depth, set to 100 and 10 in the reference paper [1] and in our baseline.

Random Forest is a strong choice for tabular security data: it needs little tuning, is insensitive to feature scaling, trains and predicts quickly, and reports which features mattered most. Its limitations are memory use for large forests and the need for numeric input, so it cannot read raw text directly.

2.4 Isolation Forest

Isolation Forest [17] is an unsupervised anomaly detector. It builds random trees that split the data on random features at random values. An anomalous point, being rare and different, is isolated after only a few splits, while a normal point needs many. The anomaly score of a point *x* is:

$$s(x, n) = 2^{-E[h(x)] / c(n)} \quad (8)$$

where E[h(x)] is the average path length needed to isolate *x* across the trees, *n* is the sample size and c(n) is the average path length for *n* points, used for normalization. Scores close to 1 indicate anomalies. A **contamination** parameter tells the method what fraction of the data to flag as anomalous. This parameter explains one of our notes on the reference paper (Section 3.2): with a contamination of 0.01 the method flags about 1% of the data, so it cannot detect most attacks when attacks form a much larger share.

2.5 Gradient Boosting

Gradient Boosting Machines [18] also combine trees, but sequentially. Each new tree is trained to correct the errors of the trees before it, and the model is built as a sum:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x) \quad (9)$$

where h_m is the tree added at step *m* and ν is the learning rate, which controls how much each tree contributes. **XGBoost** [19] is an efficient implementation that adds regularization and parallel computation; it was the best model in two of the studies reviewed [10, 11].

2.6 Artificial Neural Networks

An artificial **neuron** computes a weighted sum of its inputs and passes it through an **activation function** *f*:

$$y = f(\Sigma_i w_i x_i + b) \quad (10)$$

where w_i are the weights and *b* is the bias. Common activations are **ReLU**, f(z) = max(0, z), used in hidden layers, and the **sigmoid**, σ(z) = 1 / (1 + e^{-z}), which outputs a value between 0 and 1 and is used in the output neuron of a binary classifier. A **neural network** stacks neurons in layers; in a dense (fully connected) layer every neuron receives all outputs of the previous layer. A network with several hidden layers is called **deep** [23].

Training adjusts the weights to minimize a **loss function**. For binary classification this is the **binary cross-entropy**:

$$L = - [y \log p + (1 - y) \log(1 - p)] \quad (11)$$

where *y* is the true label and *p* the predicted probability. The gradient of the loss with respect to every weight is computed by **backpropagation** [20], and the weights are moved a small step

against it. The **Adam** optimizer [21] adapts the step size of each weight automatically. The data is processed in **batches**, and one pass over the whole training set is an **epoch**.

Two techniques limit overfitting. **Dropout** [22] switches off a random fraction of neurons at each training step, so the network cannot rely on any single neuron. **Early stopping** monitors performance on a validation part of the training data and stops training when it no longer improves, keeping the best weights.

2.7 Deep Architectures Compared in the Project

Multilayer Perceptron (MLP). A stack of dense layers. It treats the feature vector as a flat set of inputs and is the simplest deep model, which makes it the natural first comparison with Random Forest.

Convolutional Neural Network (CNN). A CNN applies small filters that slide over the input and detect local patterns, sharing the same weights at every position; pooling layers then keep the strongest responses [24]. CNNs were designed for images; in one dimension (Conv1D) they slide along a sequence.

Long Short-Term Memory (LSTM). A recurrent network reads a sequence one element at a time and keeps a hidden state that summarizes what it has read. Plain recurrent networks forget long-range information; the LSTM [25] solves this with a **cell state** controlled by three gates (input, forget and output) that decide what to add, keep and expose:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{12}$$

where c_t is the cell state, f_t and i_t are the forget and input gates, $\tilde{c}_t$ is the candidate content and $\odot$ is element-wise multiplication.

When CNN and LSTM are applied to a vector of flow features, as in several studies [9], the features are treated as a sequence in an arbitrary order. This is a known limitation: these architectures show their real strength on data that is genuinely sequential, such as text or event sequences, which is where the project's later phases use them.

Table (9): The learning methods compared in the project

Method	Type	Strength	Limitation	Role in the project
Random Forest	Supervised, tree ensemble	Accurate on tabular data, fast, little tuning, interpretable importances	Large memory for many trees; numeric input only	Baseline (Phase 1)
Isolation Forest	Unsupervised, anomaly detection	Needs no labels; detects the unusual	Depends on the contamination setting; more false alarms	Comparison with the reference paper
GBM / XGBoost	Supervised, sequential trees	Often the most accurate on tabular data	More tuning; sequential training	Comparison with the reference paper
MLP	Deep, dense layers	Simple, flexible	Needs scaling and more data; less interpretable	Deep model (Phase 1)
CNN (1D)	Deep, convolutional	Detects local patterns	Feature order arbitrary on tabular data	Deep model (Phase 1)
LSTM	Deep, recurrent	Models sequences and context	Slow to train; arbitrary order on tabular data	Deep model; bridge to text

2.8 Class Imbalance

In real traffic benign activity far outnumbers attacks, and in CIC-IDS2017 some attack classes make up less than 1% of the data. A model trained on such data is pushed towards the majority class, and its **accuracy becomes misleading**: a model that labels everything benign can exceed 80% accuracy while detecting nothing. Common remedies are giving the minority class a larger weight in the loss, resampling the training set, and generating synthetic minority examples with **SMOTE** [26]. Whatever the remedy, it must be applied to the training set only, and the evaluation must rely on precision, recall, F1 and the false positive rate, ideally reported per attack class.

2.9 Beyond a Single Threshold

The five metrics of Section 3.3 are computed at one decision threshold. Moving the threshold trades detection for false alarms, and the **ROC curve** shows this trade-off by plotting the true positive rate (recall) against the false positive rate for every threshold. The area under it (**AUC**) summarizes performance across all thresholds: 1 is perfect and 0.5 is random guessing. For an EDR system the relevant part of the curve is the region of very low false positive rate, because even a small rate produces many alerts at scale (Section 1.6). Training time and prediction time are reported alongside, since the detector must keep up with millions of events.

Summary of Section 2

Random Forest combines many decorrelated trees and is a strong, fast baseline for tabular security data. Deep networks (MLP, CNN, LSTM) learn their own representations but need careful scaling, regularization and more computation, and CNN and LSTM are designed for sequences rather than for flat feature vectors. A valid comparison requires a stratified split, statistics computed on the training data only, and metrics that remain honest under class imbalance. These are the principles on which the experiment of the current phase is built.

Section 3: Natural Language Processing and Transformers

This section introduces the methods that the project will apply in graduation project 1, where the input is no longer a vector of numbers but the text of commands and logs.

3.1 Why Text

Section 1.5 showed that the evidence of living-off-the-land, fileless and obfuscated attacks lies in the command lines and log messages of the endpoint. These are text: sequences of symbols with vocabulary, order and context. **Natural Language Processing (NLP)** is the field that gives computers the ability to represent and analyze such sequences, and its methods have proven effective on command lines and scripts [3, 5].

3.2 Representing Text

A model cannot read characters directly. Text is first split into units called **tokens**, a step called **tokenization**. Splitting into whole words fails on words never seen in training, which are frequent in commands (file names, random strings, encoded payloads). Modern models therefore use **subword tokenization**, such as WordPiece or Byte-Pair Encoding (BPE), which breaks a rare word into frequent pieces: an unseen string is still represented by pieces the model knows.

Each token is then mapped to a vector of numbers, its **embedding**. Embeddings are learned so that tokens used in similar contexts receive similar vectors. Early methods gave each word a single fixed vector; contextual models, described next, compute a different vector for the same token depending on the words around it, so that «-enc» in a PowerShell command is represented differently from the same letters inside a file name.

3.3 From Recurrent Networks to Attention

Recurrent networks, including the LSTM, read a sequence one token at a time. This has two drawbacks: the computation cannot be parallelized across the sequence, and information from distant tokens must pass through many steps, so it weakens. **Attention** removes both problems: to build the representation of a token, the model looks directly at every other token in the sequence and decides how much each one matters.

3.4 The Transformer

The **Transformer** [27] is an architecture built entirely on attention. Each token is projected into three vectors: a query (Q), a key (K) and a value (V). The output for each token is a weighted average of all values, where the weights measure how well its query matches every key — **scaled dot-product attention**:

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_k}) V \quad (13)$$

where d_k is the dimension of the keys; dividing by $\sqrt{d_k}$ keeps the values in a stable range. **Multi-head attention** runs several attention operations in parallel so that each head can capture a different kind of relation. Because attention itself ignores order, a **positional encoding** is added to each token's embedding. A Transformer layer combines multi-head attention with a small feed-forward network, residual connections and normalization, and many layers are stacked. The original design has an **encoder**, which builds representations of the input, and a **decoder**, which generates output text.

3.5 BERT

BERT (Bidirectional Encoder Representations from Transformers) [28] uses only the encoder. It is first **pre-trained** on very large amounts of text with two tasks: **masked language modeling**, in which 15% of the tokens are hidden and the model must predict them from the context on both sides, and next sentence prediction. Through pre-training the model learns general knowledge of language.

The pre-trained model is then **fine-tuned** for a specific task with a relatively small labeled dataset. For classification, a special token [CLS] is placed at the start of the input; its final representation summarizes the whole sequence, and a small classification layer on top of it is trained together with the rest of the model. The base model has 12 layers, a hidden size of 768 and about 110 million parameters, and accepts at most 512 tokens per input.

3.6 Variants Relevant to the Project

Table (10): BERT-family models relevant to the project

Model	What distinguishes it	Use in the project
BERT [28]	General pre-trained encoder	Reference model for fine-tuning on commands
CodeBERT [29]	Pre-trained on pairs of natural language and source code	Candidate for command lines and scripts, as in [5]
Sentence-BERT [30]	Produces one vector per sentence for similarity comparison	Grouping similar alerts, as in [4]
DistilBERT [31]	Distilled version: about 40% smaller and 60% faster, keeping most of BERT's accuracy	Candidate when computing resources are limited

These encoder models differ from **large language models (LLMs)** such as GPT, which are decoder models trained to generate text and have billions of parameters. For a classification task with labeled data, a fine-tuned encoder is smaller, faster and more consistent; the studies reviewed found general LLMs unreliable as standalone detectors [8] and not better than tree models on structured data [10].

3.7 Applying Transformers to Security Text

Command lines and logs differ from natural language in ways that affect every step:

- **Vocabulary.** They contain paths, flags, hashes, IP addresses, GUIDs and long encoded strings. A common practice is to **normalize** variable parts, for example replacing specific paths, numbers or addresses with placeholders, so that the model learns patterns rather than memorizing values.
- **Obfuscation.** Changes in letter case, inserted characters, string concatenation and Base64 encoding are designed to defeat signatures. Encoded content can be decoded before classification, while the obfuscation itself is kept as a signal, since benign commands rarely use it [3].
- **Length.** Scripts and long logs exceed the 512-token limit, so they must be truncated, split into windows, or compressed beforehand, the problem that HyperGLLM addresses for EDR logs [2].
- **Imbalance.** Malicious commands are a tiny fraction of real telemetry, so the remedies of Section 2.8 apply here as well [3].

On this basis, graduation project 1 will collect command-line data (for example from Sysmon process-creation events in a lab, alongside public datasets), fine-tune a BERT-family model to classify commands, and compare it with a simple baseline that represents text by token frequencies and classifies it with Random Forest.

Summary of Section 3

Transformers represent each token in the context of all others through attention, and BERT makes this power available through pre-training followed by fine-tuning on a small labeled dataset. Variants such as CodeBERT, Sentence-BERT and DistilBERT match the project's needs for code-like text, alert grouping and limited resources. Applying them to security text requires attention to unusual vocabulary, obfuscation, input length and class imbalance.

Summary of Chapter 1

The endpoint is the main target of modern attacks, and EDR systems are the tool for seeing and stopping them, but their signature-based analysis misses attacks that hide in legitimate tools and obfuscated commands and produces many false alarms. Machine learning offers detectors that learn from data: Random Forest is a strong baseline on numeric features, and deep net-

works are the candidates the current phase tests against it under fair conditions. Transformers and BERT extend learning to the text of commands and logs, where the evidence of the hardest attacks lies. These three sets of ideas are the foundations on which the project's three phases are built.

References

1. R. Abu Lail, M. Shkoukani, H. Abusaimeh, "Endpoint Detection And Response Systems (EDR) Enhancing Using Classification Techniques: A Machine Learning Approach," Int. J. Adv. Sig. Img. Sci., vol. 12, no. 2s, pp. 114–131, 2026.
2. H. Zhou, J. Pan, M. Peng, S. Huang, H. Zheng, "HyperGLLM: An Efficient Framework for Endpoint Threat Detection via Hypergraph-Enhanced Large Language Models," Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, no. 41, pp. 35094–35102, 2026. doi:10.1609/aaai.v40i41.40815
3. V. Outrata, B. Štěpánková, M. A. Polák, M. Kopp, "Command-Line Obfuscation Detection in Real-World Telemetry under Extreme Class Imbalance," Proceedings of the Second International Conference on Natural Language Processing and Artificial Intelligence for Cyber Security (NLPAICS), 2026. https://aclanthology.org/2026.nlpaics-1.8/
4. B. Štěpánková, V. Outrata, M. Kopp, "Semantic Clustering of Obfuscated Command-Line Detections for Alert Reduction," Proceedings of the Second International Conference on Natural Language Processing and Artificial Intelligence for Cyber Security (NLPAICS), 2026. https://aclanthology.org/2026.nlpaics-1.19/
5. S. Varlioglu, N. Elsayed, M. Ozer, Z. ElSayed, J. M. Emmert, "Detecting Fileless Cryptojacking in PowerShell Using AST-Enhanced CodeBERT Models," arXiv preprint arXiv:2602.18285, 2026.
6. J. Ispahany, M. R. Islam, M. A. Khan, M. Z. Islam, "iCNN-LSTM+: A Batch-Based Incremental Ransomware Detection System Using Sysmon," IEEE Access, vol. 13, pp. 87978–87998, 2025. doi: 10.1109/ACCESS.2025.3569635
7. T. Bilot, B. Jiang, Z. Li, N. El Madhoun, K. Al Agha, A. Zouaoui, T. Pasquier, "Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based Intrusion Detection Systems," 34th USENIX Security Symposium (USENIX Security 25), 2025.
8. A. Konstantinou, D. Kasimatis, W. J. Buchanan, S. U. Jan, J. Ahmad, I. Politis, N. Pitropakis, "Leveraging LLMs for Non-Security Experts in Threat Hunting: Detecting Living off the Land Techniques," Machine Learning and Knowledge Extraction, vol. 7, no. 2, art. 31, 2025. doi: 10.3390/make7020031
9. M. L. Ali, K. Thakur, S. Schmeelk, J. Debello, D. Dragos, "Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study," Applied Sciences, vol. 15, no. 4, art. 1903, 2025. doi:10.3390/app15041903
10. L. Mehavilla, M. Rodríguez, J. García, Á. Alesanco, "Evaluating large language models effectiveness for flow-based intrusion detection: a comparative study with ML and DL baselines," Artificial Intelligence Review, vol. 59, no. 2, 2026. doi:10.1007/s10462-025-11432-2
11. S. Dharel, P. K.C., R. K. Maharjan, "Comparative Analysis of Ensemble Machine Learning and Hybrid Deep Learning Models for Intrusion Detection," International Journal on Engineering Technology, vol. 3, no. 2, pp. 15–23, 2026. doi:10.3126/injet.v3i2.95492

12. J. L. Leevy, T. M. Khoshgoftaar, "A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data," Journal of Big Data, vol. 7, art. 104, 2020. doi:10.1186/s40537-020-00382-x

13. I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116, 2018.

14. A. Portnoy et al., "Towards Automatic Hands-on-Keyboard Attack Detection Using LLMs in Endpoint Detection and Remediation (EDR) Solutions," arXiv preprint arXiv:2408.01993, 2024.

15. M. Sharif, P. Datta, A. Riddle, K. Westfall, A. Bates, V. Ganti, M. Lentz, D. Ott, "DrSec: Flexible Distributed Representations for Efficient Endpoint Security," 2024 IEEE Symposium on Security and Privacy (SP), pp. 3609–3624, 2024. doi:10.1109/SP54263.2024.00145

16. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

17. F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation Forest," Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413–422, 2008.

18. J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

19. T. Chen, C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

20. D. E. Rumelhart, G. E. Hinton, R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533–536, 1986.

21. D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations (ICLR), 2015.

22. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

23. I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.

24. Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

25. S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

26. N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, "SMOTE: Synthetic Minority Oversampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

27. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.

28. J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proceedings of NAACL-HLT, pp. 4171–4186, 2019.

29. Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547, 2020.

30. N. Reimers, I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of EMNLP-IJCNLP, pp. 3982–3992, 2019.

31. V. Sanh, L. Debut, J. Chaumond, T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.

General sources mentioned in the text without numbering: the MITRE ATT&CK framework (attack.mitre.org), the Sigma rules project (github.com/SigmaHQ/sigma), the LOLBAS project (lolbas-project.github.io), and the Sysmon documentation on Microsoft Learn.